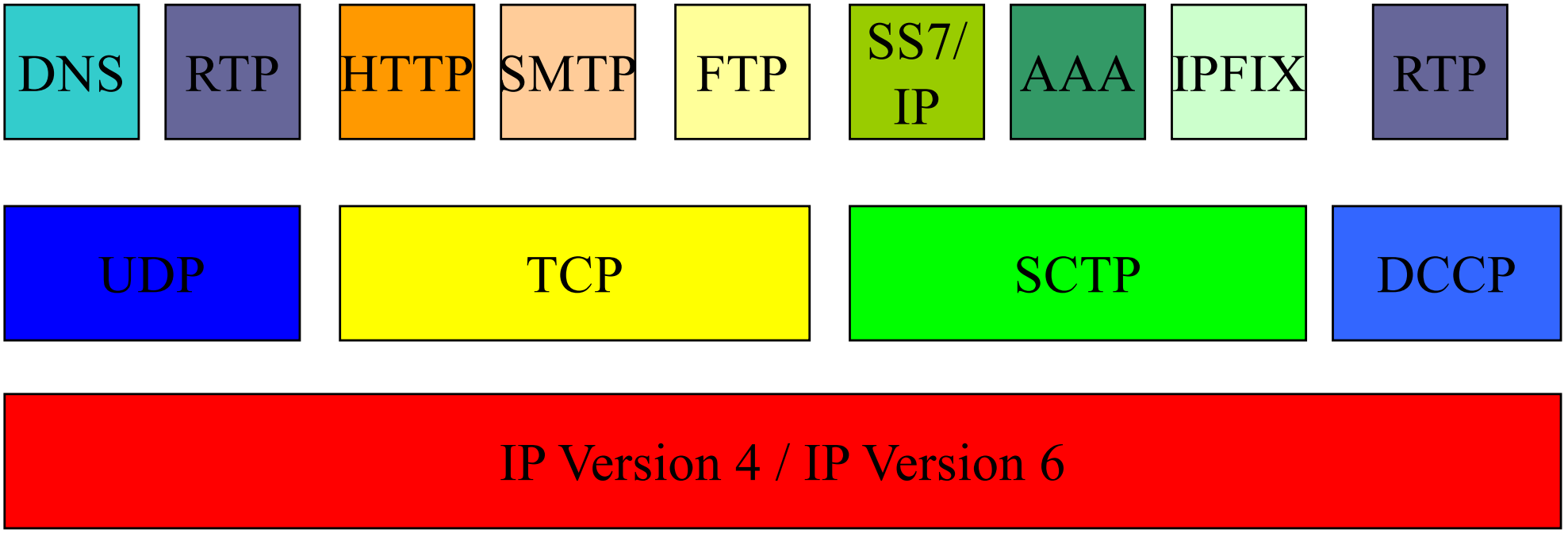
Netzwerkprogrammierung

Vorlesung 1

Literatur

- *Stevens, Fenner, Rudoff*: **Unix Network Programming**, Volume 1, 3rd Edition, Addison Wesley 2004.
- *Zahn*: **Unix-Netzwerkprogrammierung mit Threads, Sockets und SSL**, Springer 2006.
- *Comer, Stevens*: **Internetworking with TCP/IP (BSD Socket Version with Ansi C)**, Volume III, Prentice Hall 1996.
- *Kernighan, Ritchie*: **The C Programming Language**, 2nd Edition, Prentice Hall 1988.

Die Internet Protocol (IP) Protokollfamilie



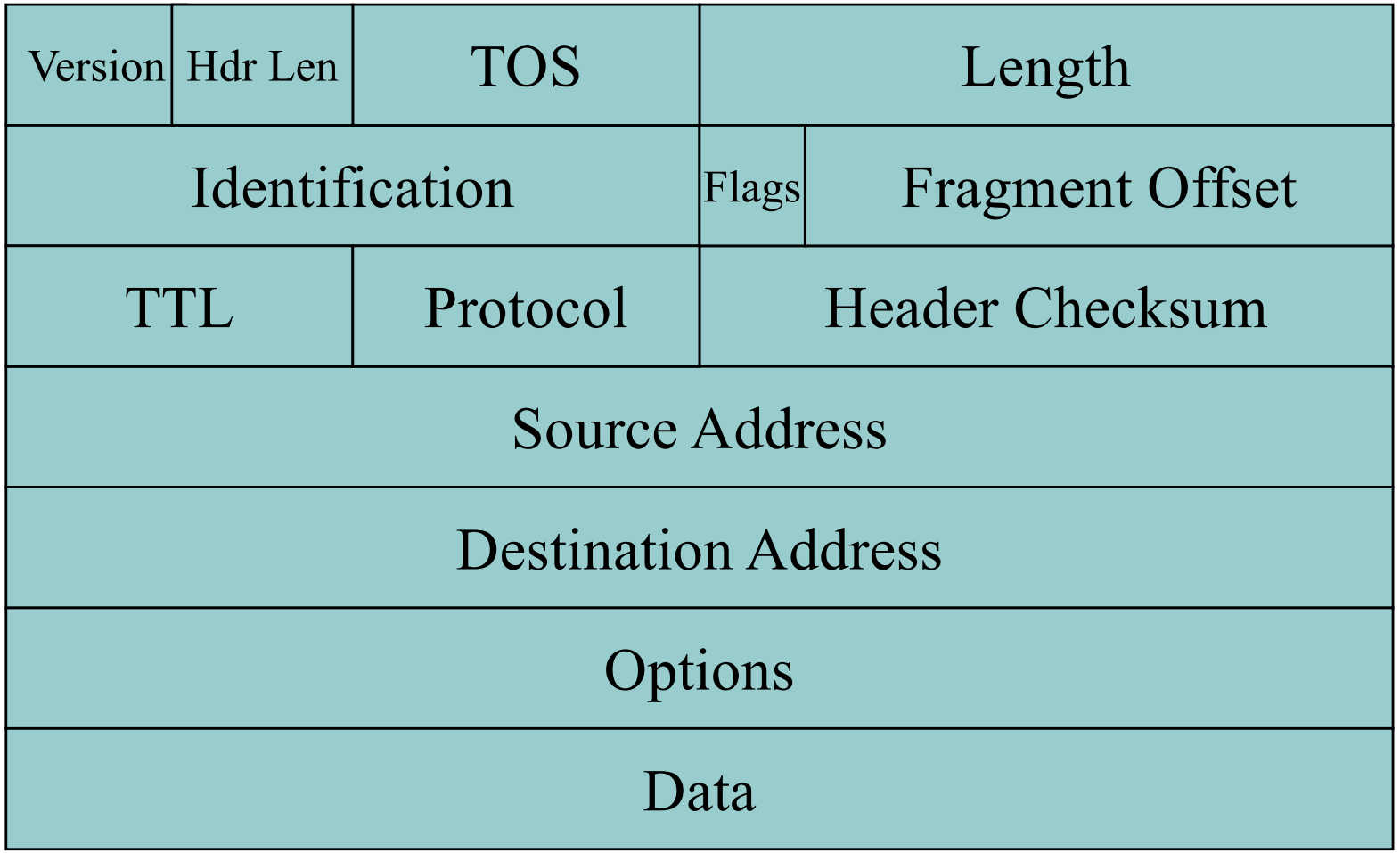
Die Socketschnittstelle

- Grenze zwischen Applikation und Betriebssystemkern.
- Verfügbar unter Unix und Windows (Winsock).
- Basis der Schnittstellen in anderen Sprachen (Java, …).
- C basiert.

Der IP-Layer (Version 4)

- Der Netzwerk-Layer der Internet Protocol Suite.
- Die Übertragung ist ungesichert, dass heißt
  - einige Pakete können verloren gehen.
  - einige Pakete können verdoppelt werden.
  - einige Pakete können in falscher Reihenfolge ankommen.
- Der IP-Layer kann bis zu 64 KByte große Pakete transportieren. Man sollte aber nur solche Paketgrößen nutzen, die nicht fragmentiert werden müssen.

IPv4 Packet


Version
Hdr Len
TOS
Length
Identification
Flags
Fragment Offset
TTL
Protocol
Header Checksum
Source Address
Destination Address
Options
Data

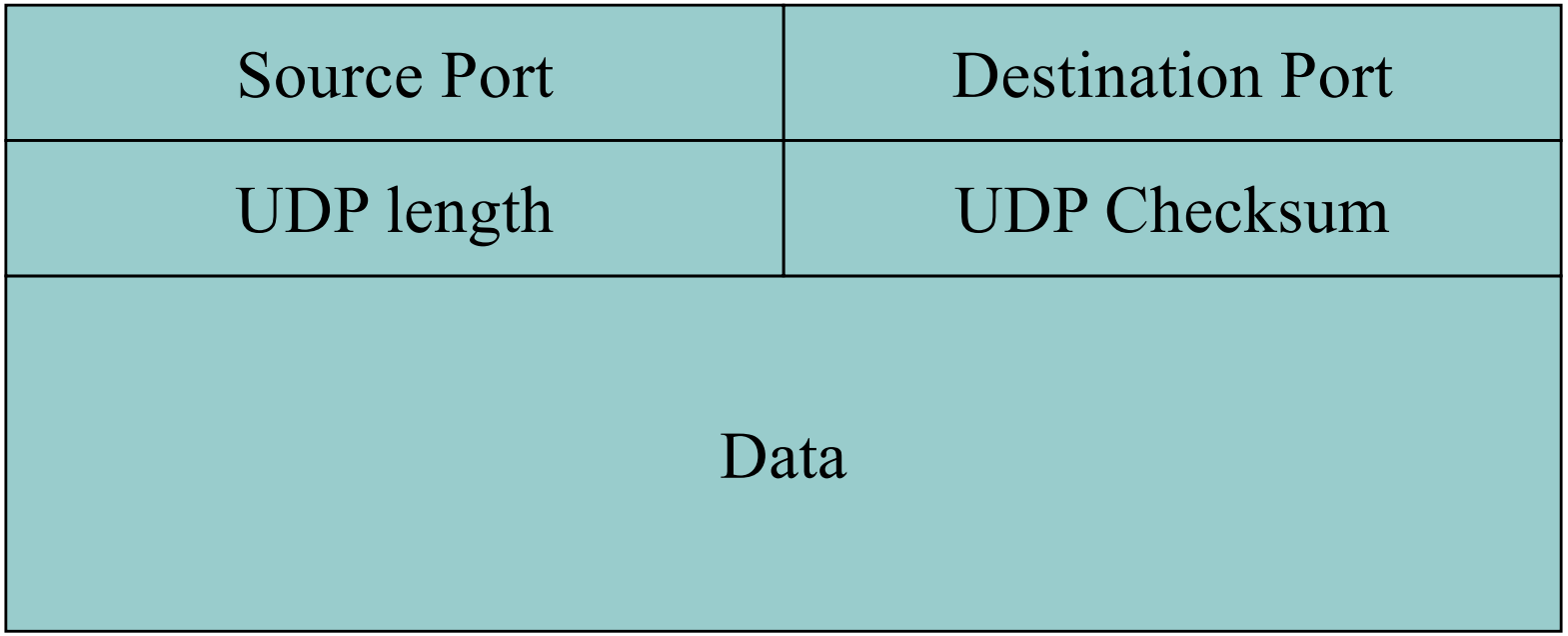
IPv4 Adressen

- Werden geschrieben als 4 Zahlen zwischen 0 und 255, durch Punkte getrennt.
- Beispiel: 192.168.1.1
- Private, Multicast, ... Genaueres RFC 6890

User Datagram Protocol: UDP

- Nachrichtenorientiert
- Nicht gesichert
- Keine Flowcontrol
- Keine Congestioncontrol
- Keine Fragmentierung
- Singlehoming
- Unicast, Multicast, Broadcast und Anycast

UDP Packet



Port Nummern

- Well Known: 0-1023
  - Echo, Discard, Telnet, DNS, ssh, http, ...
- Reserved: 1024-49151
  - MGCP, SIGTRAN, ...
- Ephemeral: 49152-65535

Transport Adresse

- Der Zielhost wird durch eine IP Adresse spezifiziert.
- Verschiedene Services werden durch unterschiedliche Portnummern identifiziert
- Transport Adresse = IP Adresse + Portnummer

Kommunikationsmodell

- Client-Server
- Peer-to-Peer

Echo Protocol für UDP

- RFC 862
- Well known Port: 7
- Sende jedes empfangene UDP Paket an den Sender zurück.
- Sicherheitsaspekte? Überlast?

Discard Protocol für UDP

- RFC 863
- Well known Port: 9
- Verwerfe jedes empfangene Paket.
- Sicherheitsaspekte? Überlast?

Daytime Protocol für UDP

- RFC 867
- Well known Port: 13
- Sende für jedes empfangene UDP Paket ein Paket mit der aktuellen Zeit an den Sender zurück.
- Sicherheitsaspekte? Überlast?

Chargen Protocol für UDP

- RFC 864
- Well known Port: 19
- Sende für jedes empfangene UDP Paket ein Paket mit einer zufälligen Länge von 0 bis 512 Byte und einem erkennbaren Muster als Inhalt an den Sender zurück. Das empfangene Paket wird verworfen.
- Sicherheitsaspekte? Überlast?

socket call

- int socket(int family, int type, int protocol);
  - family: AF_INET, AF_INET6, ...
  - type: SOCK_STREAM, SOCK_DGRAM, ...
  - protocol: 0, IPPROTO_UDP, IPPROTO_TCP, ...
- Gibt einen Filedescriptor >= 0 zurück oder im Fehlerfall -1.

bind call

- int bind(int fd, const struct sockaddr *addr,
  socklen_t addrlen);
  - fd: Filedescriptor
  - addr: Zeiger auf lokale Transportadresse
  - addrlen: Länge der lokalen Transportadresse
- Gibt 0 zurück, falls alles geklappt hat, -1 im Fehlerfall.

recvfrom call

- ssize_t recvfrom(int fd, void *buf, size_t buflen,
  int flags, struct sockaddr *from,
  socklen_t *addrlen)
  - fd: Filedescriptor
  - buf: Zeiger auf Speicherbereich
  - buflen: Länger des Speicherbereiches
  - from: Zeiger auf Ursprungstransportadresse
  - addrlen: Zeiger auf Länge der Ursprungstransportadresse (Value-Result argument)
- Gibt die Anzahl der gelesenen Bytes zurück, -1 im Fehlerfall.

sendto call

- ssize_t sendto(int fd, void *buf, size_t buflen,
  int flags, const struct sockaddr *to,
  socklen_t addrlen)
  - fd: Filedescriptor
  - buf: Zeiger auf Speicherbereich
  - buflen: Länger des Speicherbereiches
  - to: Zeiger auf Zieltransportadresse
  - addrlen: Länge der Zieltransportadresse
- Gibt die Anzahl der an den Kernel übergebenen Bytes zurück, -1 im Fehlerfall.

close call

- int close(int fd);
  - fd: Filedescriptor
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

Strukturen

- ```
  struct sockaddr {
    u_char  sa_len;                 /* total length */
    u_char  sa_family;              /* address family */
    char    sa_data[14];            /* actually longer; address value*/
  };                                /* from sys/socket.h */
  ```
- ```
  struct sockaddr_in {
    u_char  sin_len;
    u_char  sin_family;
    u_short sin_port;
    struct  in_addr sin_addr;
    char    sin_zero[8];
  };                                /* from netinet/in.h */
  ```
- ```
  struct in_addr {
    in_addr_t s_addr;
  };                                /* from netinet/in.h */
  ```
  ```

Network Byte Order

- int16 Werte brauchen 2 Bytes im Speicher.
- Ein High Order Byte, ein Low Order Byte.
- Litte Endian: Low Order Byte steht an niedrigerer Adresse.
- Big Endian: Low Order Byte steht an höherer Adresse.
- Big Endian = Network Byte Order.

Byte Order Conversion

- uint16_t htons(uint16_t);
- uint16_t ntohs(uint16_t);
- uint32_t htonl(uint32_t);
- uint32_t ntohl(uint32_t);

Echo Server

```
int main()
{
    int fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len;
    int len;
    char buf[1<<16];
    fd  = socket(AF_INET, SOCK_DGRAM, 0);

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family      = AF_INET;
    server_addr.sin_len         = sizeof(struct sockaddr_in);
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    server_addr.sin_port        = htons(7);
    bind(fd, (const struct sockaddr *) &server_addr, sizeof(server_addr));

    while (1) {
        addr_len = sizeof(client_addr);
        memset(&client_addr, 0, sizeof(client_addr));
        len      = recvfrom(fd, (void *) buf, sizeof(buf), 0,
                            (struct sockaddr *) &client_addr, &addr_len);
        sendto(fd, (const void *) buf, len, 0,
               (const struct sockaddr *) &client_addr, addr_len);
    }
}
```

Elementare Fehlerbehandlung

- Ersetze socket(fd, type, protocol) durch
- int Socket(int fd, int type, int protocol)

```
{
        int n;
        if ((n=socket(fd, type, protocol)) < 0) {
                perror(„socket“);
                exit(-1);
        }
        return(n);
}
```